Wie gut skaliert Parallelität?

Ein Multicore-Speedup-Vergleich von Rust, Go, Java, C++ und Scala

Kamilo Knezevic · Stefan Wild

LV Wissenschaftliches Arbeiten und Schreiben

Agenda

1. Motivation und Forschungsfrage
2. Methodik: Algorithmen, Sprachen, Messdesign
3. Ergebnisse
4. Diskussion
5. Conclusio

Motivation

- Taktraten steigen kaum noch, Leistung kommt über **mehr Kerne**
- Aber: 8 Kerne ≠ 8-fache Geschwindigkeit
- Jede Sprache bietet **eigene Parallel-Modelle** mit eigenen Kosten
- Üblicher Vergleich: wer ist *schneller*? Wir fragen: wer **skaliert** besser?

Forschungsfrage

Wie stark profitieren Rust, Go, Java, C++ und Scala vom Wechsel von sequentiellem zu parallelem Code auf mehreren Kernen, und hängt das von der Art des Problems ab?

1. Wie skaliert jede Sprache bei **Rechenarbeit, Tasks und Speicherlast**?
2. **Wo geht Effizienz verloren** (sequentieller Anteil, Overhead, Speicher)?
3. Ist die Sprache mit dem **besten Speedup** auch die **schnellste**?

Drei Algorithmen

	Algorithmus	Parallelisierung
Datenparallel	Primzahlen in [0, 15 Mio.] per Probedivision	Chunks à 10.000 Zahlen, Worker holen dynamisch
Taskparallel	`fib(45)` rekursiv, Cutoff 30	`fib(n-1)` als Task, `fib(n-2)` selbst, dann Join
Speicherlastig	Merge Sort, 50 Mio. 64-Bit-Zahlen	linke Hälfte als Task, Merge sequentiell mit Puffer

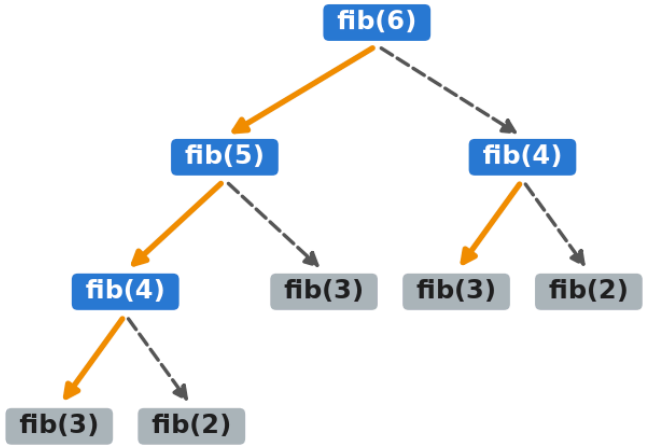
Alle Sprachen: gleiche Eingaben, gleiche Task-Anzahl (≈ 1.600 bei Fib), gleiche Ergebnisprüfung.

Algorithmus 1: Primzahlen zählen (datenparallel)



- **Aufgabe:** Primzahlen bis 15 Millionen zählen, jede Zahl einzeln prüfen
- **Idee:** Zahlen sind unabhängig → Zahlenstrahl in **Päckchen (Chunks)** schneiden und verteilen
- **Wer fertig ist, holt das nächste Päckchen:** keiner wartet, obwohl große Zahlen länger dauern

Algorithmus 2: Fibonacci (taskparallel)



Wie liest man das?

fib(n−1) wird als Task an einen freien Worker gegeben

fib(n−2) rechnet derselbe Worker selbst weiter

n ≥ 4 teilt weiter auf (Cutoff noch nicht erreicht)

n < 4 Cutoff erreicht: rechnet alleine zu Ende, ohne neue Tasks

Danach: Der Worker wartet auf den Task und addiert beide Ergebnisse (Join).

Im Experiment: fib(45) mit Cutoff 30 → rund 1.600 Tasks

(Bild: kleines Beispiel fib(6) mit Cutoff 4)

- **Aufgabe:** fib(n) = fib(n−1) + fib(n−2), rekursiv und absichtlich rechenintensiv
- **Idee:** Eine Teilaufgabe geht **an einen Helfer (Task)**, die andere macht man selbst, dann addiert man
- **Cutoff:** Ganz unten kostet Weitergeben mehr als Selbstrechnen, dort rechnet jeder allein weiter

Algorithmus 3: Merge Sort (speicherlastig)



- **Idee:** 50 Mio. Zahlen halbieren, Stücke sortieren, dann paarweise **zusammenmischen (Merge)**
- **Je weiter oben, desto weniger Merges** laufen gleichzeitig. Der letzte läuft über **alle** Daten auf **einem** Worker
- Dazu viel Speicherverkehr: jeder Merge legt einen Puffer an

Fünf Sprachen, fünf Parallel-Modelle

Sprache	Datenparallel	Taskparallel
Rust	Rayon Parallel Iterator	`rayon::join`
Go	Goroutines + atomarer Zähler	Goroutine + Channel
Java	`ForkJoinPool.invokeAll`	`RecursiveTask`
C++	OpenMP `parallel for`	OpenMP `task` / `taskwait`
Scala	`Future` je Chunk	`Future`, nicht blockierender Join

Rust, Go, Java, Scala: Work-Stealing-Scheduler · C++: OpenMP-Laufzeit (libgomp)

Messdesign

- **Baseline:** echter sequentieller Code, **nicht** der parallele Code mit 1 Worker
- Worker: **0** (sequentiell), 1, 2, 4, 8 (= physische Kerne)
- **10 Runden**, jede Konfiguration pro Runde in **frischem Prozess**, Reihenfolge **zufällig**
- Gemessen wird **inklusive Start der Threads** (gehört zu den Kosten von Parallelität)
- Jedes Ergebnis wird gegen den sequentiellen Wert **geprüft**

Kennzahlen und Statistik

- **Speedup** S(p) = Median T_{seq} / Median T_p
- **Effizienz** E(p) = S(p) / p
- **Serial Fraction** (Karp-Flatt): Anteil der Laufzeit, der sich **wie sequentiell** verhält · 0 % = ideal
- **95 %-Bootstrap-Intervalle** des Medians
- **cv** (Streuung) > 5 % = verrauscht · Ausreißer werden **nie** entfernt

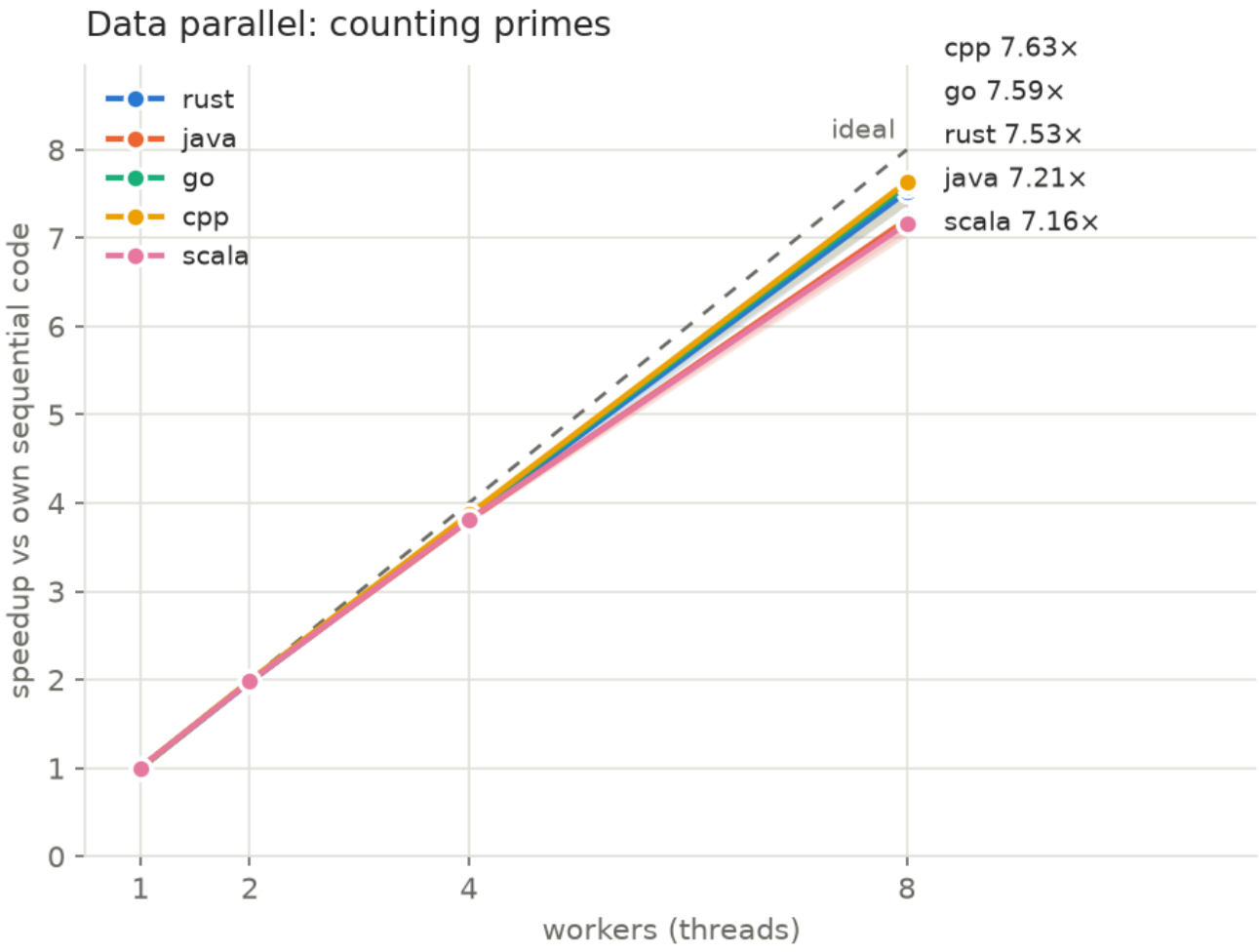
Testsysteme

	Linux (Hauptdatensatz)	Windows
CPU	Ryzen 7 5800X (Desktop)	Ryzen 7 5800HS (Laptop)
Kerne	8 phys. / 16 logisch	8 phys. / 16 logisch
OS	Ubuntu 24.04	Windows 11
Java	OpenJDK 21	Oracle JDK 23
C++	g++ 13.3	MinGW g++ 13.2
Rust · Go	1.95 · 1.25	1.99 · 1.26

Beide: libgomp für OpenMP, Rayon 1.12, Scala 3.3.6 · Gleiche Größen, 10 Runden

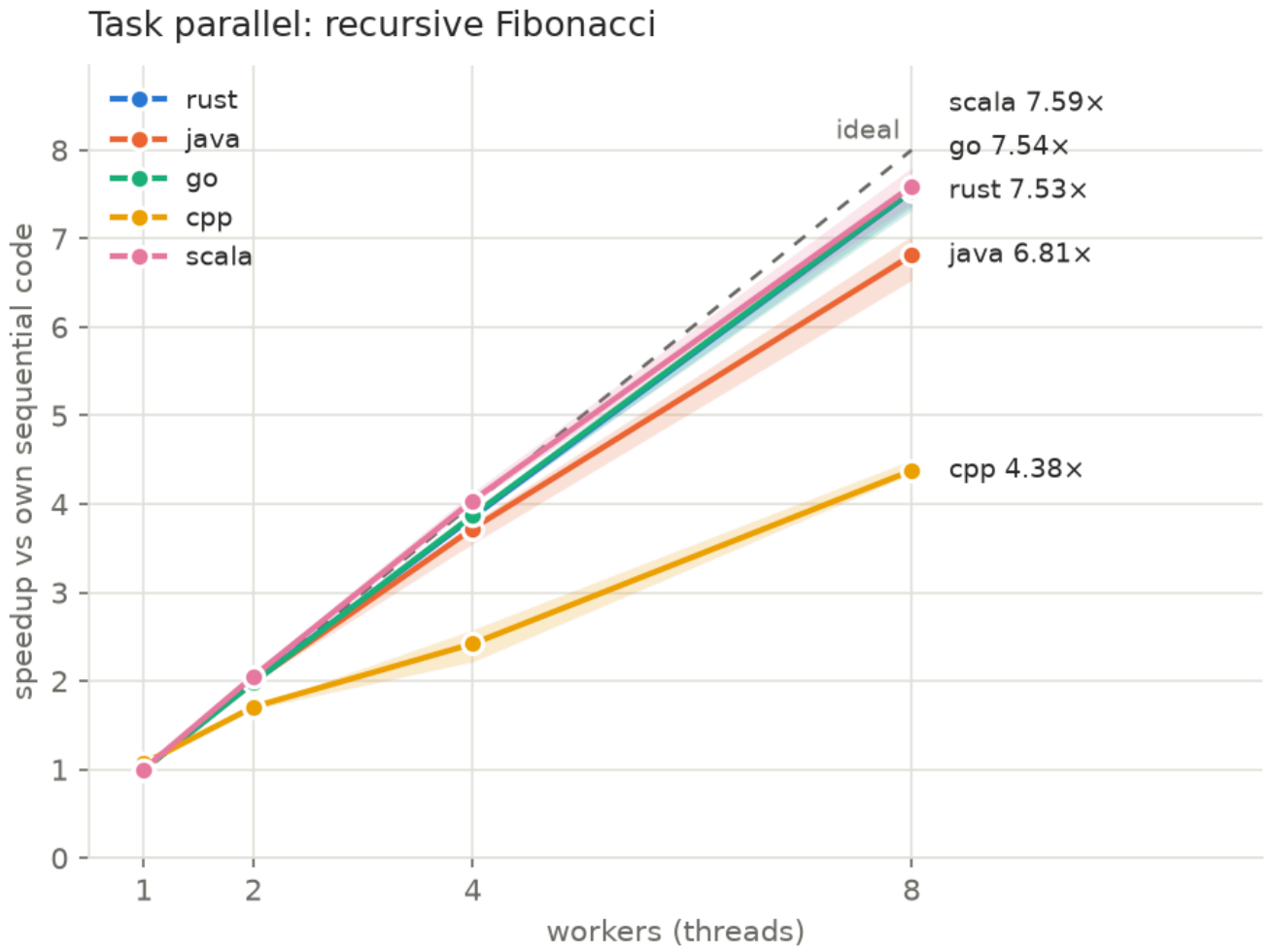
Ergebnis 1: Datenparallel (Primes)

- Alle Sprachen **7,2 – 7,6×** bei 8 Workern (Effizienz 90 – 95 %)
- Kaum Unterschiede, JVM-Sprachen minimal schwächer



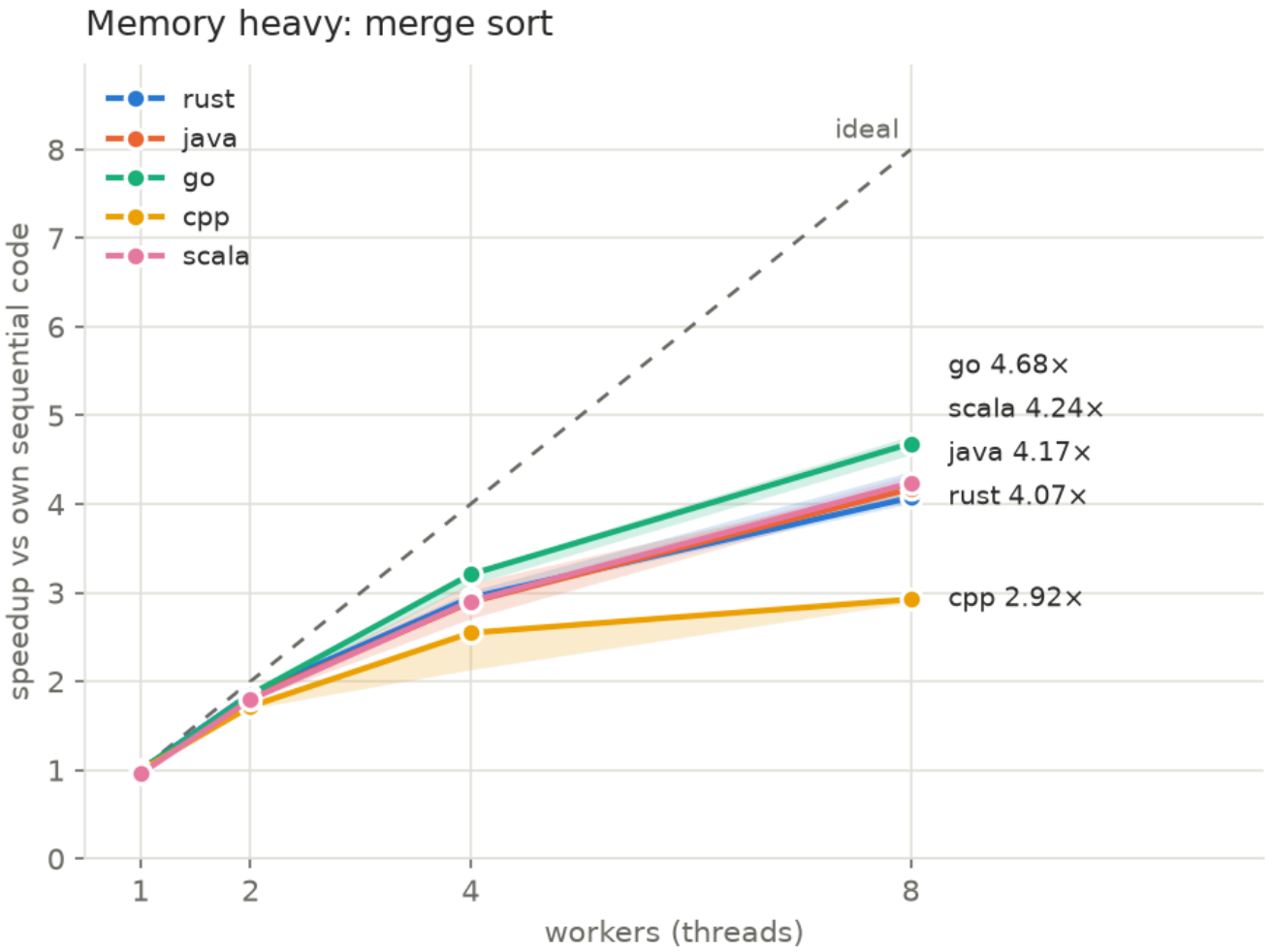
Ergebnis 2: Taskparallel (Fib)

- Rust, Go, Scala **7,5 – 7,6×**, Java **6,8×**
- **C++ nur 4,4×** (Serial Fraction 12 – 22 %)

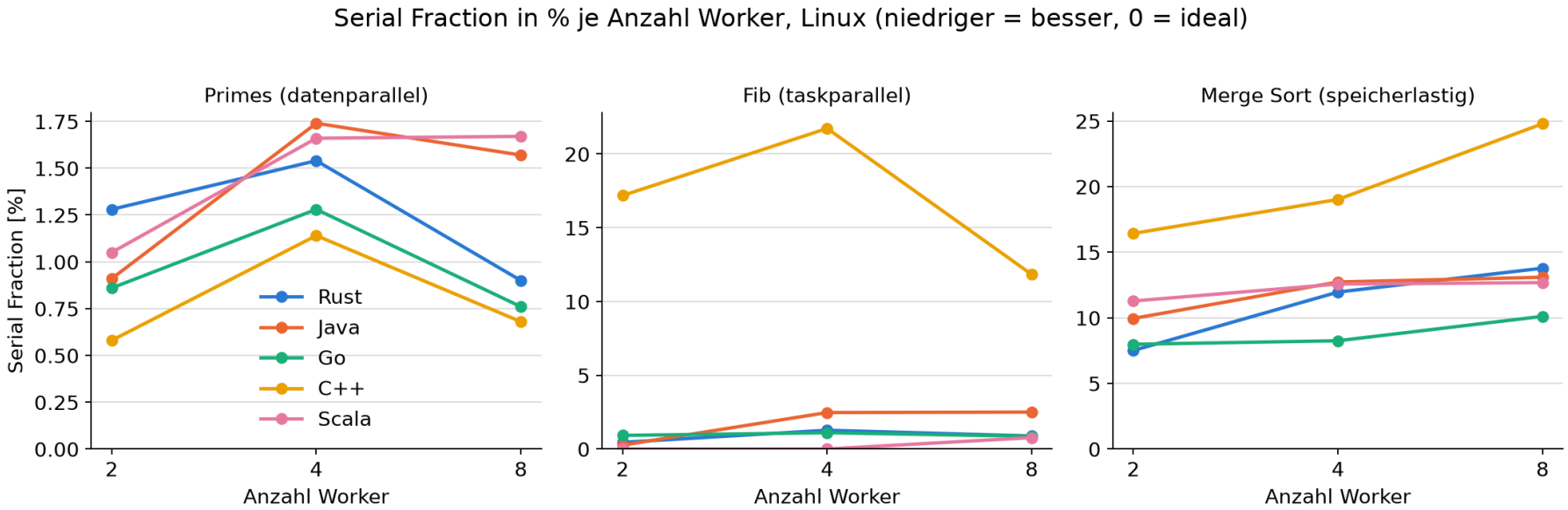


Ergebnis 3: Speicherlastig (Merge Sort)

- Nur **2,9 – 4,7×** bei 8 Workern (Effizienz 37 – 59 %)
- Go 4,7 · Scala 4,2 · Java 4,2 · Rust 4,1 · **C++ 2,9**



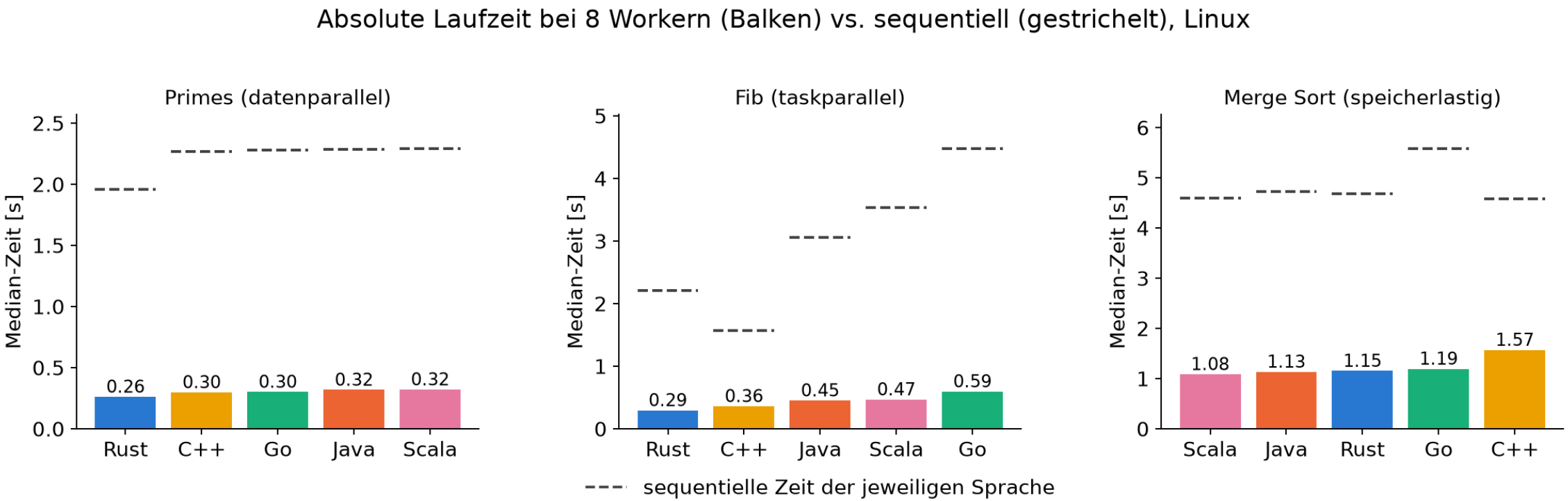
Wo geht Effizienz verloren? (Serial Fraction)



Serial Fraction = Anteil der Arbeit, der sich so verhält, als könnte er nicht parallel laufen. **0 % = ideal**, je höher, desto mehr Verlust.

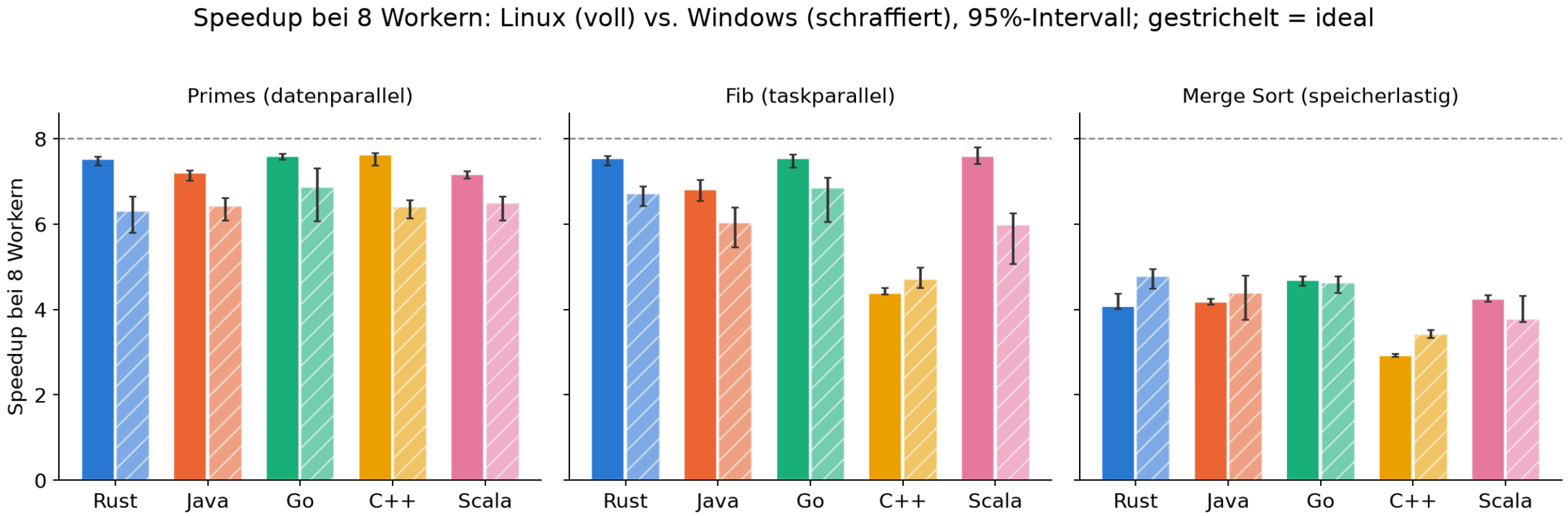
- **Primes ≈ 1 %:** fast alles läuft wirklich parallel
- **Merge Sort 8 – 25 %, steigt mit den Workern:** letzte Merges laufen einzeln, Speicher wird zum Engpass
- **C++-Fib 12 – 22 %** schon ab 2 Workern: Task-Verwaltung kostet viel (Ursache offen)

Absolute Laufzeit: wer ist am schnellsten?



- **Rust** bei Primes und Fib vorn · **Scala/Java** bei Merge Sort vorn
- **Go-Fib:** 7,5× Speedup, aber absolut Letzter · **C++:** sequentiell top, bei 8 Workern Letzter (Msort)

Windows vs. Linux



- Gleiches **Muster**, aber Speedup **5 – 15 % niedriger** und mehr Rauschen (cv bis 16 %)
- Primes Effizienz 79 – 86 % statt 90 – 95 %

Diskussion

- **Merge Sort:** Rechnerisch höchstens ≈ **4,9×** bei 8 Kernen (letzte Merges laufen einzeln), gemessen **bis 4,7×**: das Problem setzt die Grenze, nicht die Sprache
- **C++-Tasks:** auf beiden Systemen reproduzierbar, Ursache **nicht geklärt** (Vermutung: OpenMP-Task-Verwaltung)
- **Praxis:** Zuerst den **Aufgabentyp** prüfen, bevor man Kerne zählt. Speedup immer **zusammen mit der absoluten Zeit** lesen
- **Grenzen:** je Typ ein Problem, nur 8 Kerne, eine CPU-Familie. Gleiches **Design** heißt nicht optimierter Code je Sprache

Conclusio

Frage: Wie stark profitieren die Sprachen von mehr Kernen?

- Hängt vom **Problemtyp** stärker ab als von der **Sprache**: Primes ≈ 7,5× · Fib 4,4 – 7,6× · Msort 2,9 – 4,7×
- **Speedup ≠ Geschwindigkeit:** Go-Fib skaliert 7,5×, ist absolut aber Letzter
- **Ausreißer:** C++ bei Tasks (4,4×) und Merge Sort (2,9×)
- Beste Sprache gibt es nicht pauschal: **Rust** schnellster bei CPU-Arbeit, **JVM-Sprachen** knapp vorn bei Speicherlast